1. `download.py` expliqué

Nutri-Arbitre — fiche de révision · étape 1 sur 5 du trajet des données (download → clean → tools → garde_fou → agent)

> **En une phrase :** ce fichier va chercher sur Internet la liste des céréales du petit-déjeuner vendues en France (Open Food Facts) et l'enregistre dans `data/raw_cereales.json`.
> **Lancer :** `python -m nutri.download --pages 20`

Lignes 15 à 18 : les réglages

```
15 URL = "https://world.openfoodfacts.org/api/v2/search"
16 USER_AGENT = "nutri-arbitre/0.1 (projet etudiant Coding for AI)"
17 FIELDS = "code,product_name,brands,labels_tags,nutriscore_grade,nutriments"
18 SORTIE = Path("data/raw_cereales.json")
```

Ce sont des **constantes** : des valeurs fixes, écrites en MAJUSCULES par convention.

- `URL` : l'adresse de l'API Open Food Facts. Une **API**, c'est un site fait pour les programmes plutôt que pour les humains : il répond en données brutes (JSON) au lieu d'une page web.
- `USER_AGENT` : la « carte de visite » du programme. Open Food Facts demande qu'on se présente, pour savoir qui fait les requêtes.
- `FIELDS` : les seules informations qu'on demande (code-barres, nom, marque, labels, Nutri-Score, valeurs nutritionnelles). Inutile de télécharger les photos ou les ingrédients.
- `SORTIE` : le fichier où on enregistre.

`telecharger_page` (lignes 21 à 40) : récupérer une page

```
21 def telecharger_page(page: int, page_size: int = 100) -> list[dict]:
23     params = {
24         "categories_tags_en": "breakfast-cereals",
25         "countries_tags_en": "france",
26         "fields": FIELDS,
27         "page": page,
28         "page_size": page_size,
29     }
31     for essai in range(5):
32         try:
33             r = requests.get(URL, params=params, headers={"User-Agent":
USER_AGENT}, timeout=60)
34             r.raise_for_status()
35             return r.json().get("products", [])
36         except requests.RequestException as e:
37             attente = 15 * (essai + 1)
38             print(f"page {page} : erreur (...), nouvel essai dans {attente} s")
39             time.sleep(attente)
40     raise RuntimeError(f"page {page} : échec après 5 essais")
```

L'API ne renvoie pas tout d'un coup : elle découpe les résultats en **pages** de 100 produits, comme les pages de résultats de Google.

- `params` est un dictionnaire qui décrit ce qu'on cherche : la catégorie « céréales du petit-déjeuner », le pays « France », le numéro de la page.
- `requests.get(...)` envoie la requête, comme quand on tape une adresse dans le navigateur.
- `raise_for_status()` déclenche une erreur si le serveur répond « problème » (par exemple le code 503 : « serveur surchargé »).
- `r.json().get("products", [])` lit la réponse et renvoie la liste des produits, ou une liste vide s'il n'y en a pas.
- **La boucle `for essai in range(5)` avec `try / except`** : on essaie au maximum 5 fois. Si ça échoue (`except`), on attend 15 s, puis 30, 45… avant de recommencer. Si ça réussit, `return` sort immédiatement de la fonction. Si les 5 essais échouent, on abandonne avec un message clair (`raise RuntimeError`).

`sauver` (lignes 43 à 45) : écrire le fichier

```
43 def sauver(produits: list[dict]) -> None:
44     SORTIE.parent.mkdir(exist_ok=True)
45     SORTIE.write_text(json.dumps(produits, ensure_ascii=False), encoding="utf-8")
```

- `mkdir(exist_ok=True)` crée le dossier `data/` s'il n'existe pas, sans erreur s'il existe déjà.
- `json.dumps(...)` transforme la liste Python en texte JSON ; `write_text` écrit ce texte dans le fichier.

`main` (lignes 48 à 58) : le chef d'orchestre

```
48 def main(nb_pages: int) -> None:
49     produits = []
50     for page in range(1, nb_pages + 1):
51         lot = telecharger_page(page)
52         print(f"page {page} : {len(lot)} produits")
53         if not lot:  # plus aucun produit : on s'arrête
54             break
55         produits.extend(lot)
56         sauver(produits)  # on écrit après chaque page
57         time.sleep(7)     # respecter la limite de l'API
```

On demande les pages une par une, de 1 à 20 :

- si une page est vide, il n'y a plus de produits et on s'arrête (`break`) ;
- sinon on ajoute ses produits à la liste (`extend`) et on enregistre ;
- `time.sleep(7)` fait une pause de 7 s, parce qu'Open Food Facts limite à environ 10 recherches par minute. Ça montre qu'on respecte les règles du service.

Lignes 61 à 64 : le lancement

```
61 if __name__ == "__main__":
62     parser = argparse.ArgumentParser()
63     parser.add_argument("--pages", type=int, default=20)
64     main(parser.parse_args().pages)
```

`if __name__ == "__main__"` veut dire « seulement si on lance ce fichier directement » (`python -m nutri.download`), et pas quand un autre fichier l'importe. `argparse` lit l'option `--pages 20` tapée dans le terminal.

> **Ce qui s'est vraiment passé (30 octobre 2026) :** le serveur a renvoyé plusieurs erreurs 503. Les pages 4, 6, 7, 9 et 10 ont réussi après un ou deux nouveaux essais. La page 11 a échoué 5 fois de suite, donc le script s'est arrêté. Comme on enregistre après chaque page, les 10 premières pages (1 000 produits) étaient déjà dans le fichier. Après nettoyage : **953 céréales**.
> **Limite à connaître :** Open Food Facts renvoie les produits dans son propre ordre. Nos 1 000 produits ne sont donc pas forcément représentatifs de toutes les céréales vendues en France (par exemple, 57 % sont bio).

Questions d'examinateur

> 1. Pourquoi attendre 7 secondes entre deux pages ?
> 2. Que se passe-t-il si le serveur répond « 503 » ?
> 3. Pourquoi enregistre-t-on après *chaque* page et pas une seule fois à la fin ?
> 4. Pourquoi ne demande-t-on que certains champs (`FIELDS`) ?
>
> Essaie d'y répondre avec tes mots avant de lire le corrigé à la page suivante.

Corrigé

1. **7 secondes :** Open Food Facts autorise environ 10 recherches par minute, soit une toutes les 6 s. Avec 7 s, on reste sous la limite : on respecte le service gratuit et on évite d'être bloqué.
2. **Erreur 503 :** `raise_for_status()` déclenche une erreur, que `except` attrape. Le programme attend (15 s, puis 30, 45…) et réessaie, jusqu'à 5 fois. Après 5 échecs, il s'arrête avec un message clair au lieu de continuer avec des données manquantes.
3. **Enregistrer après chaque page :** si le téléchargement s'arrête en route, on garde ce qui est déjà arrivé. C'est exactement ce qui s'est passé le 30 octobre : la page 11 a échoué, mais les 1 000 produits des pages 1 à 10 étaient sauvés.
4. **Seulement certains champs :** une fiche produit complète contient des centaines d'informations (photos, ingrédients, emballage…). On ne demande que ce dont on a besoin : le téléchargement est plus rapide, le fichier plus petit, et on charge moins le serveur.